

《NixOS 系統配置文件結構完全指南》

從入門到企業級架構的實戰教學

作者：OpenAI ChatGPT 版本：Draft v0.1

前言

Linux 世界長期存在一個問題：

系統配置會隨著時間逐漸失控。

你可能曾經遇過：

- 不知道當初修改了哪些設定
- 套件版本互相衝突
- 重灌系統後環境無法重現
- Server 與 Laptop 配置不一致
- 更新後系統壞掉
- 不敢升級 production 環境
- 不知道哪個 shell script 改動了系統

NixOS 的核心目標，就是解決這些問題。

它不是「另一個 Linux 發行版」而已。

它更像：

- Infrastructure as Code
- 可重現系統
- 宣告式作業系統
- 版本化配置管理
- 原子化部署平台

本書將從完全初學者角度出發，逐步建立：

1. NixOS 思維模型
2. Nix 語言基礎
3. configuration.nix 架構
4. 模組化配置設計
5. flakes 現代化配置
6. 多主機管理
7. 企業級 NixOS 架構
8. GitOps 與 CI/CD

本書不是單純「指令手冊」。

而是一本：

「如何設計可維護 NixOS 系統架構」的工程實戰書。

第一篇：理解 NixOS

第1章：NixOS 到底是什麼？

1.1 傳統 Linux 的問題

在多數 Linux 發行版中：

```
sudo apt install nginx
sudo vim /etc/nginx/nginx.conf
sudo systemctl restart nginx
```

這種方式有幾個問題：

1. 系統狀態不可追蹤
2. 修改歷史容易遺失
3. 無法完整重現
4. 不容易 rollback
5. 配置散落在各處

隨著時間增加：

系統會逐漸變成「只有原作者知道怎麼運作」的狀態。

這就是：

「Configuration Drift」

1.2 NixOS 的解法

NixOS 使用：

「宣告式配置」

你不是手動修改系統。

而是：

「描述系統應該長什麼樣子」。

例如：

```
{
  services.nginx.enable = true;

  users.users.alice = {
    isNormalUser = true;
    extraGroups = [ "wheel" ];
  };
}
```

這代表：

- 啟用 nginx
- 建立 alice 使用者
- 加入 sudo 群組

然後：

```
sudo nixos-rebuild switch
```

NixOS 會：

1. 計算新的系統狀態
2. 建立新的 generation
3. 原子化切換
4. 保留 rollback 能力

1.3 第一個重要觀念：系統是「可建構的」

在 NixOS 中：

你的作業系統不是一堆被手動修改的檔案。

而是：

「由 configuration.nix 建構出來的結果」。

因此：

```
configuration.nix
  ↓
Nix Evaluation
  ↓
Derivations
  ↓
```

System Closure
↓
Bootable System

這是理解 NixOS 最重要的核心。

Lab 1：安裝你的第一個 NixOS VM

目標

本 Lab 將建立：

- 第一個 NixOS 虛擬機
- 初次理解 configuration.nix
- 初次使用 nixos-rebuild

建議環境

工具	建議
Hypervisor	VirtualBox / VMware / KVM
RAM	4GB 以上
CPU	2 Core
Disk	30GB

安裝步驟

Step 1：下載 ISO

前往：

- NixOS Official ISO

建議版本：

- GNOME Installer
- 最新 Stable Release

Step 2：建立 VM

建議：

設定	值
RAM	4096MB
CPU	2
Disk	30GB
EFI	Enabled

Step 3：完成安裝

安裝完成後：

```
cat /etc/nixos/configuration.nix
```

你會看到：

```
{ config, pkgs, ... }:  
  
{  
  imports = [  
    ./hardware-configuration.nix  
  ];  
  
  networking.hostName = "nixos";  
  
  time.timeZone = "Asia/Taipei";  
  
  users.users.demo = {  
    isNormalUser = true;  
    extraGroups = [ "wheel" ];  
  };  
  
  environment.systemPackages = with pkgs; [  
    vim  
    git  
  ];  
  
  system.stateVersion = "25.05";  
}
```

這就是：

「你的整個系統定義」。

第2章：理解 configuration.nix

2.1 configuration.nix 的角色

在 NixOS 中：

```
/etc/nixos/configuration.nix
```

是：

「系統主配置入口」。

它負責：

- 載入 modules
- 定義 services
- 定義 users
- 安裝 packages
- 配置 bootloader
- 設定 networking

你可以把它想成：

```
main() of your operating system
```

2.2 最小可用 configuration.nix

```
{ config, pkgs, ... }:  
  
{  
  system.stateVersion = "25.05";  
}
```

這是理論上的最小系統。

實際上：

Installer 會自動幫你產生更多設定。

2.3 imports 機制

```
imports = [  
  ./hardware-configuration.nix  
];
```

代表：

「把其他 module 載入目前 configuration」。

這是 NixOS 模組化的基礎。

之後你會大量使用：

```
configuration.nix  
  ↓  
services.nix  
networking.nix  
desktop.nix  
users.nix
```

教學範例：拆分你的第一個 Module

原始配置

```
{  
  networking.hostName = "nixos";  
  
  services.openssh.enable = true;  
  
  users.users.alice = {  
    isNormalUser = true;  
    extraGroups = [ "wheel" ];  
  };  
}
```

Step 1：建立 users.nix

```
{  
  users.users.alice = {  
    isNormalUser = true;
```

```
    extraGroups = [ "wheel" ];  
  };  
}
```

Step 2：修改 configuration.nix

```
{  
  imports = [  
    ./users.nix  
  ];  
  
  networking.hostName = "nixos";  
  
  services.openssh.enable = true;  
}
```

Step 3：重新建構

```
sudo nixos-rebuild switch
```

這就是：

「NixOS 模組化配置」的第一步。

第3章：Nix 語言基礎

3.1 為什麼一定要學 Nix 語言？

因為：

NixOS 本質上：

不是 YAML。

不是 JSON。

而是一種：

「Functional Configuration Language」。

因此：

理解 Nix 語言 = 理解 NixOS 。

3.2 Attribute Set

Nix 最重要的資料結構：

```
{  
  name = "alice";  
  age = 20;  
}
```

很像：

- JSON object
 - Python dict
 - JavaScript object
-

3.3 List

```
[ "git" "vim" "curl" ]
```

常用於：

```
environment.systemPackages = [  
  pkgs.git  
  pkgs.vim  
  pkgs.curl  
];
```

3.4 Function

```
name: "Hello ${name}"
```

呼叫：

```
(name: "Hello ${name}") "Alice"
```

結果：

```
Hello Alice
```

Lab 2：建立你的第一個開發環境

目標

建立：

- git
 - vim
 - curl
 - htop
 - tree
-

Step 1：修改 configuration.nix

```
environment.systemPackages = with pkgs; [  
  git  
  vim  
  curl  
  htop  
  tree  
];
```

Step 2：重新建構

```
sudo nixos-rebuild switch
```

Step 3：驗證

```
git --version  
vim --version  
htop
```

第4章：Nix Store 與 Immutable System

4.1 /nix/store 是什麼？

查看：

```
ls /nix/store
```

你會看到：

```
/nix/store/abc123-vim-9.1  
/nix/store/xyz789-git-2.50
```

這裡是：

NixOS 的核心。

4.2 為什麼套件名稱前面有 hash？

因為：

Nix 使用：

「完整依賴關係計算」。

任何配置變化：

都會產生新的 store path。

因此：

系統是 immutable 的。

4.3 Immutable 的優勢

優勢 1：不會覆蓋舊版本

你可以同時存在：

```
vim-9.0  
vim-9.1
```

優勢 2：安全 rollback

```
sudo nixos-rebuild switch --rollback
```

優勢 3：可重現

相同配置：

得到相同系統。

教學案例：打造模組化桌面配置

目標架構

```
/etc/nixos/  
├─ configuration.nix  
├─ hardware-configuration.nix  
├─ desktop.nix  
├─ packages.nix  
├─ services.nix  
└─ users.nix
```

configuration.nix

```
{ config, pkgs, ... }:  
  
{  
  imports = [  
    ./hardware-configuration.nix  
    ./desktop.nix  
    ./packages.nix  
    ./services.nix  
    ./users.nix  
  ];  
  
  system.stateVersion = "25.05";  
}
```

desktop.nix

```
{
  services.xserver.enable = true;

  services.displayManager.sddm.enable = true;

  services.desktopManager.plasma6.enable = true;
}
```

packages.nix

```
{ pkgs, ... }:

{
  environment.systemPackages = with pkgs; [
    firefox
    vscode
    git
    vim
  ];
}
```

services.nix

```
{
  services.openssh.enable = true;

  services.printing.enable = true;
}
```

users.nix

```
{
  users.users.alice = {
    isNormalUser = true;

    extraGroups = [
      "wheel"
      "networkmanager"
    ]
  }
}
```

```
};  
};  
}
```

第5章：Flakes 現代化架構

5.1 為什麼 Flakes 很重要？

傳統 NixOS 配置有幾個問題：

- dependency 不固定
- channel 狀態不一致
- 無法精準重現
- 多主機管理困難

flakes 的目標：

「讓 NixOS 配置真正工程化」。

5.2 第一個 flake.nix

```
{  
  description = "My NixOS Configuration";  
  
  inputs = {  
    nixpkgs.url = "github:NixOS/nixpkgs/nixos-25.05";  
  };  
  
  outputs = { self, nixpkgs }: {  
    nixosConfigurations.myhost = nixpkgs.lib.nixosSystem {  
      system = "x86_64-linux";  
  
      modules = [  
        ./configuration.nix  
      ];  
    };  
  };  
}
```

5.3 建構系統

```
sudo nixos-rebuild switch --flake .#myhost
```

Lab：建立多主機配置

目標

管理：

- laptop
- desktop
- server

使用同一份 repository。

專案結構

```
nixos-config/  
├── flake.nix  
├── hosts/  
│   ├── laptop/  
│   ├── desktop/  
│   └── server/  
├──  
├── modules/  
│   ├── desktop/  
│   ├── server/  
│   ├── networking/  
│   └── users/  
├──  
└── profiles/  
    ├── base.nix  
    ├── desktop.nix  
    └── server.nix
```

第6章：Home Manager

6.1 為什麼需要 Home Manager？

NixOS 管理：

「系統層」。

但很多設定屬於：

「使用者層」。

例如：

- zshrc
- git config
- neovim
- tmux
- vscode
- shell aliases

Home Manager 用來：

「宣告式管理使用者環境」。

6.2 第一個 Home Manager 配置

```
{
  programs.git = {
    enable = true;

    userName = "Alice";

    userEmail = "alice@example.com";
  };

  programs.zsh.enable = true;
}
```

第7章：NixOS 的真正威力

當你學會：

- module system
- flakes
- overlays
- Home Manager
- deployment
- secrets management

你會開始理解：

NixOS 並不是「Linux distro」。

它更像：

「可程式化作業系統平台」。

企業案例：統一管理 100 台機器

傳統 Linux：

每台機器手動配置
↓
配置逐漸漂移
↓
維護困難

NixOS：

Git Repository
↓
CI/CD
↓
Immutable Deployment
↓
All Machines Reproducible

最終專案：建立完整 Homelab

本書最後將實作：

- Router
- NAS
- Kubernetes Node
- Monitoring
- Reverse Proxy
- Git Server
- CI/CD
- Secrets Management
- GitOps Deployment

全部使用：

- flakes
- deploy-rs
- sops-nix
- Home Manager
- reusable modules

建立完整：

「企業級 NixOS 基礎設施」。

學習路線建議

初學者階段

目標：

- 熟悉 configuration.nix
- 能安裝 packages
- 能管理 services

建議章節：

1. Part 1
 2. Part 2
 3. 基本 Labs
-

中階階段

目標：

- 模組化配置
- flakes
- Home Manager
- 多主機管理

建議章節：

1. Part 3
 2. Part 4
 3. Part 5
-

高階階段

目標：

- overlays
- deployment
- secrets
- GitOps
- enterprise architecture

建議章節：

1. Part 6
2. Part 7
3. Part 8

本書特色

本書將避免：

- 單純貼配置
- 沒有架構脈絡
- 沒有工程思維
- 缺少實戰案例

而是聚焦：

1. 為什麼這樣設計
2. 如何拆分架構
3. 如何維護大型配置
4. 如何多人協作
5. 如何部署 production
6. 如何建立可重現環境

下一步

建議接下來依序撰寫：

1. 完整 Part 1
2. 完整 Labs
3. 所有配置案例
4. flakes 深度篇
5. deployment 與 GitOps
6. enterprise architecture

之後可進一步輸出：

- mdBook
- PDF
- EPUB
- GitHub Pages
- DevOps training materials